A Study on Three Linear Assignment Algorithms for Single-Cell Data Integration

Robert wan

Introduction

Single-cell data integration, a rapidly evolving and important field of research, promises to unlock transformative biological discoveries by unifying the analysis of at single-cell resolution across diverse experimental datasets. The fundamental challenge lies in accurately mapping individual cells that have been profiled using different single-cell sequencing technologies, such as scRNA-seq for transcriptomic analysis and scATAC-seq for chromatin accessibility, even through these datasets exhibit variations in their underlying data structure, inherent technical biases, and pervasive biological noise. While single-cell technologies offer high resolution in characterizing cellular states and functions at the individual cell level, the considerable levels of technical and biological noise inherent in these measurements often obscure the true underlying biological signal, significantly complicating the matching process. Consequently, integrating data from multiple modalities to obtain a comprehensive view of cellular biology remains a challenge. Current state-of-the-art methods achieve an accuracy of only 20%.

Our current research focuses on developing a Gibbs sampler-type algorithm to address the single-cell data integration problem. While promising, this approach suffers from long computation times and has suboptimal performance. An alternative and promising framework is the class of linear assignment algorithms. These algorithms are generally known for their ability to produce optimal solutions for assignment problems, and some are quite fast for reasonably-sized datasets. To explore the potential of this alternative framework in the context of single-cell data integration, we conducted a series of simulation studies. These studies were designed to evaluate the performance of three popular linear assignment algorithms – the Hungarian algorithm, the Jonker-Volgenant algorithm, and the auction algorithm – across a range of scenarios. These scenarios include variations in different levels of noise of the data, the number of cells being analyzed, and the number of genes or features measured for each cell. The aim of this investigation was to gain a thorough understanding of the strengths and limitations of these linear assignment algorithms in tackling the complexities of single-cell data integration.

Background

In this section, we briefly introduce the linear assignment problem (LAP) and provide high-level descriptions of the three algorithms we evaluated.

All LAPs concern matching elements of multiple sets. The most classical variant of LAP seeks to find a one-to-one matching between elements of two sets while minimizing the total cost. The formal statement can be written as follows.

Let C represent the cost matrix where c_{ij} is the cost of matching element i in the first set to element j in the second set. Let X represent the assignment matrix where x_{ij} represents assigning element i in the first set to element j in the second set. Our goal is to

$$\text{minimize} \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij}$$

$$\text{subject to} \sum_{j=1}^{n} x_{ij} = 1,\ \forall i = 1,..., n$$

$$\text{and } x_{ij} \in \{0, 1\},\ \forall i, j = 1,..., n.$$

The LAP framework extends nicely to single-cell integration. We can view the two sets as data from two modalities, where each element represents a cell. Our goal is to match cells across modalities while minimizing some cost. We are free to define cost in any method, for example, by using the values of each feature to calculate the correlation between cells across modalities.

One of the foundational methods to solve the LAP is the Hungarian algorithm. This combinatorial optimization algorithm solves the assignment problem in polynomial time, typically with a time complexity of O(n4). Its core strategy involves transforming the cost matrix through row and column reductions, subtracting minimum values to introduce zeros without altering the optimal assignment. It then seeks to find a maximum matching using only zero-cost edges in the resulting bipartite graph representation. If a perfect matching covering all n elements is found using only zero-cost assignments, it represents the optimal solution. Otherwise, the algorithm iteratively adjusts the matrix and potential dual variables (node prices) until an optimal zero-cost assignment is achieved.

A faster alternative is the Jonker-Volgenant algorithm, with a worst-case complexity of $O(n^3)$. The Jonker-Volgenant algorithm solves a dual problem of the Hungarian algorithm and utilizes a shortest augmenting path approach. It iteratively searches for paths in the graph representation that can augment the current matching while considering the costs. By avoiding the costly operation of finding edges, it often converges more quickly than the traditional Hungarian method, making it suitable for large-scale assignment tasks.

The auction algorithm offers a different, intuitive approach based on economic principles. It simulates an auction where elements in one set are “bidders” while elements in the other set are “items”. The cost matrix represents how much each bidder values each item. Bidders continuously bid in increments for items based on their value and the current price. Objects increase their prices based on the bids received. This iterative bidding process continues until a stable assignment is reached where assigned pairs satisfy certain equilibrium conditions. A key advantage of the auction algorithm is that it is well-suited for parallel computing architectures.

Methods

We conducted a simulation study to evaluate the performance of three algorithms under varying conditions. The simulation was designed to mimic a real setting in which our goal is to establish 1-on-1 matches between samples from two different but related datasets.

Data Generation and Objective Function

We generated two base matrices, A and B, which we may conceptually think of as representing single-cell RNA-seq data and single-cell ATAC-seq data. The base matrix A contains 100 rows and 25 columns, which we can think of as 100 samples with 25 features each. The value of each element is drawn independently from a standard normal distribution. We created matrix B by adding random noise to each element of matrix A, where noise is drawn independently from a normal distribution with zero mean. This ensures that each row in matrix B is correlated to the same row in matrix B. We then shuffled the row order of matrix B to create matrix B' in order to simulate disrupted row correspondence. During the simulations, the algorithms would attempt to uncover the true row order by switching the rows of B' to maximize the correlation.

We defined the cost matrix as the correlation between matrices A and B'. The Hungarian algorithm and the Junker-Volgenant algorithm minimizes cost and requires cost to be positive. To work around this, we transformed each element of the cost matrix with:

$$\widetilde{Cost}_{i,j} = max(Cost) - Cost_{i,j},$$

where $max(Cost)$ is the maximum value of the cost matrix. This transformation does not change the optimality structure, and minimizing $\widetilde{Cost}_{i,j}$ is equivalent to maximizing $Cost_{i,j}$.

Scenarios, Metrics, and Procedure

We assessed the performance of the three algorithms by changing three variables.

1. **Variable 1: Noise.** We changed the variance of the noise distribution to study how each algorithm would perform when matrices A and B became dissimilar. A larger variance of noise makes the matrix B more different from matrix A. The different values of variance we used here were 0.5, 1, 1.5, 2, 2.5, 3, 5, and 10.
2. **Variable 2: Number of Observations.** We varied the number of rows in matrices A and B to study how increasing the number of samples may impact matching accuracy and computation time. The numbers of rows we used here were 50, 100, 200, 500, and 1,000.
3. **Variable 3: Number of Features.** We varied the number of columns in matrices A and B to study how increasing the number of features may impact matching accuracy. The numbers of columns we used here were 10, 25, 50, 100, and 200.

We only change one variable at a time while fixing the other two variables at their base values, which were 2 for noise, 100 for the number of rows, and 25 for the number of features. For each scenario, we generated the data and ran the algorithms 1,000 times. We then evaluated two metrics:

1. **Correctness:** the proportion of rows that are correctly matched.
2. **Computation time:** the time it takes to run the algorithm.

Results

Overall, all three algorithms demonstrated comparable correctness.The Jonker-Volgenant algorithm produced results identical to the Hungarian algorithm. This is expected because Jonker-Volgenant is an optimized version of Hungarian. The auction algorithm produced slightly different results. This can be attributed to the epsilon hyperparameter, which we conveniently set to 1 over the number of samples without further fine-tuning. We anticipate the performance of the auction algorithm to converge to that of Hungarian and Jonker-Volgenant as we further decrease epsilon.

Noise had a large impact on matching correctness (Figure 1). All three algorithms achieved 100% correctness over all iterations when noise was drawn from $N(0,\ 0.5)$ and over most iterations when noise was draw from from $N(0,\ 1)$. However, when noise overpowered the signal (i.e. when the variance of noise was larger than the variance of the data generating distribution), correctness decreased. The reaffirms one reason that makes single-cell data integration challenging - the high noise-to-signal ratio.

The number of observations also influenced matching correctness. Increasing the number of samples led to a decrease in correctness. This suggests that larger datasets pose greater challenges to matching accuracy. In contrast, increasing the number of features improved correctness, as more information was available for the algorithms to calculate correlations. When we had as many features as the number of samples, the result was almost fully accurate.

Overall, Jonker-Volgenant consistently had the fastest computation time, achieving speeds of up to 30 times faster than the Hungarian algorithm when dealing with a large number of observations. The auction algorithm's performance varied, running slower than Hungarian for smaller datasets but surpassing it as the number of observations increased. It is worth noting that the computation time of the auction algorithm would also depend on the epsilon hyperparameter, with a trade-off between speed and accuracy. Runtime decreased with an increase in the number of features, suggesting that the algorithms took less time to converge when given abundant information for matching.

Discussion

Our simulations demonstrated that LAP algorithms may be promising for single-cell data integration. The Hungarian, Jonker-Volgenant, and auction algorithms demonstrated high accuracy and computational efficiency. Notably, they outperformed the Gibbs sampler-type algorithm we developed. For instance, the LAP methods matched 100 x 25 matrices with noise sampled from $N(0,\ 1)$ in seconds with almost 100% accuracy. On the other hand, the Gibbs sampler-type algorithm took five minutes to achieve 40% accuracy when noise was sampled from $N(0,\ 0.3)$.

Despite such strong performance in simulations, applying these LAP algorithms effectively to real-world single-cell data faces a hurdle. We simulated the matrices to have maximum correlation when rows were correctly matched, so using the correlation matrix as the cost matrix was valid. However, constructing a biologically accurate cost function that captures true cell similarities across complex, noisy experimental datasets remains a major, unsolved challenge. While LAP algorithms are promising tools, further research into creating appropriate cost functions is essential for their successful application in real-world single-cell integration.

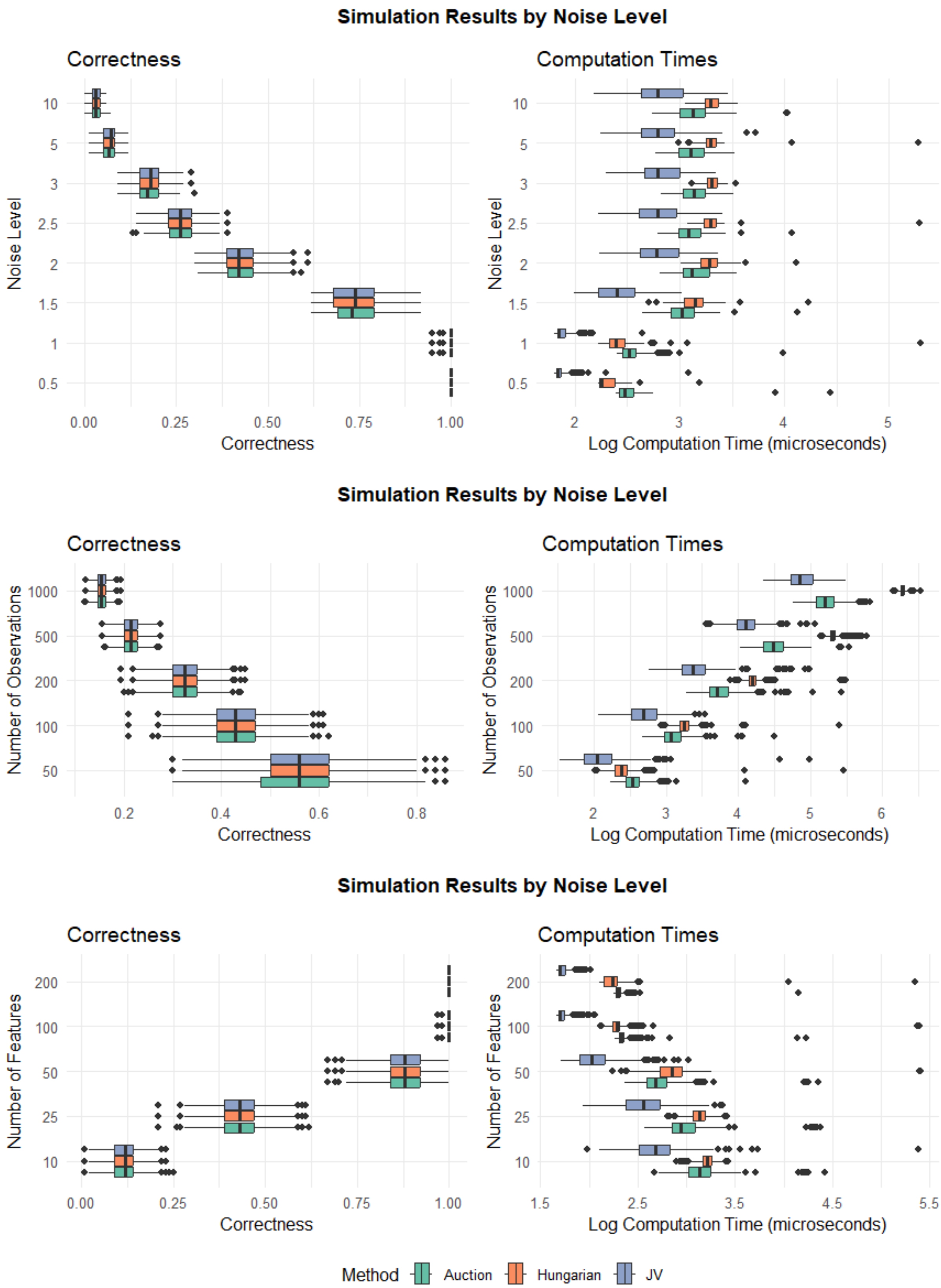

Figure 1. Matching correctness and computation time for different simulation scenarios.